

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

S6-in-class

Linked List Processing: Flattening, Subsets, and Permutations

LINKED LISTS Functions of Python §2

The LINKED LISTS functions are a set of predeclared functions provided in Python §2 for llist processing. Please refer to the [online reference](#) for details of these LINKED LISTS functions.

Problems:

1. **[In-class only]** The task for this question will be given by your Avenger during the Studio session.

Write a function called `flatten_one_level` that takes a llist whose elements are either numbers or llists of numbers (not nested more deeply than that), and returns a single flat llist containing all the numbers, in order. **Use reduce in your function.**

```
def flatten_one_level(lst):
    ...
```

Example call:

```
flatten_one_level(llist(1, llist(2, 3), 4, llist(5, 6)))
# Result: llist(1, 2, 3, 4, 5, 6)
```

2. **[In-class only]** In this question, llists represent *sets*. Each element of the set appears exactly once in its llist representation, and the order does not matter. So the llist `llist(1, 2, 3)` represents the same set as the llist `llist(3, 2, 1)`.

In this question, you are supposed to compute all subsets of a given set. Your function `subsets` takes as argument an llist, representing the given set, and needs to return an llist of llists, each representing a unique subset of the given set.

```
def subsets(xs):
    ...
```

Example call:

```
subsets(llist(1, 2, 3))
# Result: llist(llist(),
#              llist(1), llist(2), llist(3),
#              llist(1,2), llist(1,3), llist(2,3),
#              llist(1,2,3))
```

3. **[In-class only]** A *permutation* of a llist s is a llist with the same elements as s , but in a possibly different order. For example, the llist `llist(3, 1, 2)` is a permutation of `llist(1, 2, 3)`. Write a function `permutations` that takes a llist s as argument and returns a llist of all permutations of s .

```
def permutations(s):  
    ...
```

Example call:

```
permutations(llist(1, 2, 3))  
# Result: llist(llist(1, 2, 3), llist(1, 3, 2),  
#           llist(2, 1, 3), llist(2, 3, 1),  
#           llist(3, 1, 2), llist(3, 2, 1))
```